WorkLink: A Real-Time AI-Powered Service Marketplace for Blue-Collar Workforce Enablement

1st M. Vennela
Department of Computer Science and Engineering
Gayatri Vidya Parishad College of Engineering (Autonomous)
Roll No: 322103310156
322103310156@gvpce.ac.in
Mobile: 9550542861

2nd N. L. Satwika
Department of Computer Science and Engineering
Gayatri Vidya Parishad College of Engineering (Autonomous)
Roll No: 322103310169
322103310169@gvpce.ac.in
Mobile: 9515163160

3rd P. Pawan
Department of Computer Science and Engineering
Gayatri Vidya Parishad College of Engineering (Autonomous)
Roll No: 322103310179
322103310179@gvpce.ac.in
Mobile: 7981975044

4th Sk. Khadir
Department of Computer Science and Engineering
Gayatri Vidya Parishad College of Engineering (Autonomous)
Roll No: 323103310L21
323103310l21@gvpce.ac.in
Mobile: 9966004403

Project Guide: Ms. G. Vani
Assistant Professor, Department of Computer Science and Engineering
Gayatri Vidya Parishad College of Engineering (Autonomous)
Visakhapatnam, India
vanig@gvpce.ac.in

***Abstract*—Digital service marketplaces have transformed urban service access, yet workers in the unorganized blue-collar segment still face fragmented demand discovery, language barriers, delayed trust establishment, and weak digital identity signaling. This paper presents WorkLink, an end-to-end AI-enabled web platform that connects customers and verified local workers through role-aware workflows, geospatial matching, escrow-style payment flow, real-time communication, and multilingual assistance. The system is engineered as a MERN architecture with Socket.IO-driven real-time channels, Stripe-backed payment events, Cloudinary media handling, and a Gemini-powered assistant for contextual support. In contrast to conventional listing portals, WorkLink emphasizes transaction reliability through OTP-based job start verification, stateful job lifecycle management, and transparent notifications for both customers and workers. The implementation is supported by architecture, lifecycle, payment, and data-model artifacts and is validated through integrated feature-level testing across job posting, assignment, communication, payment confirmation, and completion proof flows. The project demonstrates a practical blueprint for trustworthy, inclusive, and scalable on-demand service delivery.**

***Index Terms*—Service Marketplace, MERN Stack, Real-Time Systems, Geospatial Matching, OTP Verification, AI Assistant, Stripe Payments, Blue-Collar Digitization.**

I. Introduction

The growth of digital platforms has improved convenience for consumers but has not uniformly benefited informal service workers such as plumbers, electricians, cleaners, and technicians. A major gap persists between customer intent and worker discoverability, especially in local and language-diverse contexts. Existing classified and listing platforms typically stop at contact exchange, leaving pricing, trust, execution monitoring, and payment settlement weakly structured.

WorkLink addresses this gap by introducing a full-stack transaction workflow that converts user intent into structured service execution. The platform includes role-separated user journeys, contextual AI support, real-time communication, and trust checkpoints from request creation to payment closure. The design combines engineering rigor and practical usability, focusing on real constraints faced by workers and customers.

The project is motivated by four core observations: (i) workers need guided digital interaction and language support; (ii) customers need transparent progress and reliable completion evidence; (iii) service assignment requires more than static listing and must consider skills, proximity, and availability; and (iv) secure payment release must be tied to job state transitions. WorkLink operationalizes these principles through backend workflow controls, event-driven notifications, and UI-level accessibility.

A. Problem Statement

Current local-service discovery systems exhibit three recurring limitations:

- **Unstructured trust establishment:** no robust mechanism to verify physical job start and completion.
- **Weak transactional integrity:** payment and completion are often disconnected, increasing disputes.

- **Limited worker enablement:** multilingual support, guided communication, and contextual digital assistance are insufficient.

Hence, there is a need for an integrated marketplace platform that enforces lifecycle consistency, supports real-time collaboration, and improves accessibility for digitally diverse users.

B. Objectives

The primary objectives of WorkLink are:

- Build a production-style full-stack marketplace for customer-worker service transactions.
- Implement trust-preserving lifecycle controls using OTP verification and evidence-backed completion.
- Enable geospatial and skill-based assignment through algorithmic matching.
- Support real-time chat, notifications, and live progress visibility.
- Integrate secure payment capture and release through webhook-driven state transitions.
- Improve usability via multilingual UI and AI-assisted interactions.

C. Contributions

This work makes the following contributions:

1) A complete role-aware marketplace architecture integrating AI, payments, geospatial matching, and real-time channels.
2) A lifecycle-centric job model with explicit state transitions for assignment, travel, OTP start, execution, and completion.
3) A practical trust framework combining verification, chat persistence, media evidence, and dispute-oriented event logging.
4) Deployment-ready engineering artifacts, including architecture and ER diagrams, payment lifecycle flow, and notification flow.

II. Literature Survey

On-demand marketplaces have evolved from directory models to managed transaction ecosystems. Matching quality and assignment fairness are commonly modeled as optimization tasks, where bipartite assignment methods such as the Hungarian algorithm remain foundational [1], [2]. In practical web systems, low-latency event exchange is often implemented through WebSocket-based channels for interactive communication and status synchronization [6].

Trust and security in digital transactions rely on identity, tokenized sessions, and auditable verification workflows. JSON Web Tokens (JWT) are widely used for secure claims transfer in session design [3], while password-hardening techniques such as bcrypt mitigate credential risk [4]. For payment-critical workflows, event-driven confirmation through webhooks is a standard engineering pattern [9].

Recent intelligent service interfaces increasingly use large multimodal models for contextual assistance, query refinement, and intent structuring [5]. However, many platforms still underutilize AI for inclusive support in multilingual and low-digital-literacy scenarios. WorkLink positions AI as an assistive layer rather than a replacement of transactional controls.

III. Proposed System

WorkLink follows a modular client-server architecture built on React (frontend), Node.js/Express (backend), and MongoDB (persistence), with Socket.IO for real-time communication and external integrations for AI, payments, and media.

A. Architecture Overview

The architecture separates concerns across interaction, orchestration, persistence, and integration layers. Frontend modules include role-specific dashboards, chat, map tracking, and language toggling. Backend services expose REST endpoints, lifecycle controllers, matching services, and webhook handlers. External services include Gemini API for assistant features, Stripe for payments, and Cloudinary for media assets.

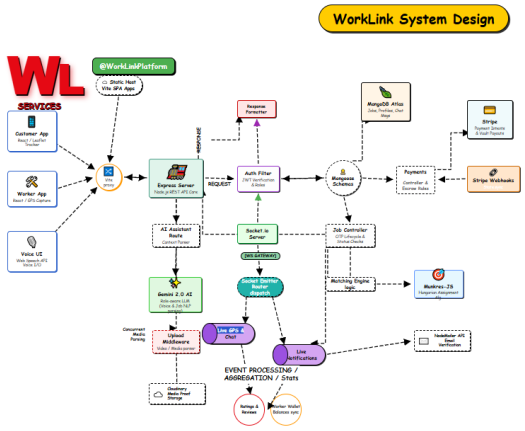

Fig. 1. Overall WorkLink system architecture with client, backend services, and external integrations.

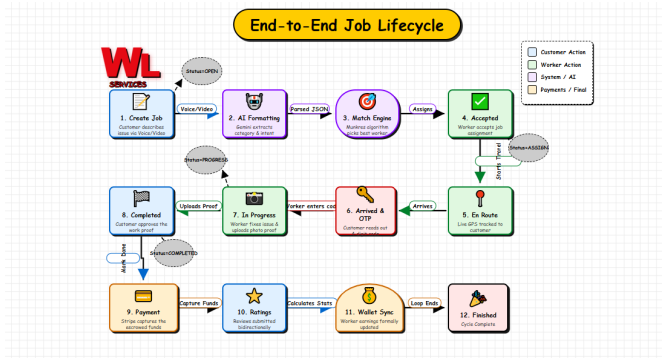

Fig. 2. High-level lifecycle flow in WorkLink from job posting to closure.

B. Data Model and Entity Relations

The core entities are `User`, `WorkerProfile`, `Job`, `ChatRoom`, `ChatMessage`, `Payment`, `Rating`, and `Notification`. GeoJSON-based location fields support proximity filtering, while lifecycle fields and verification metadata ensure state consistency.

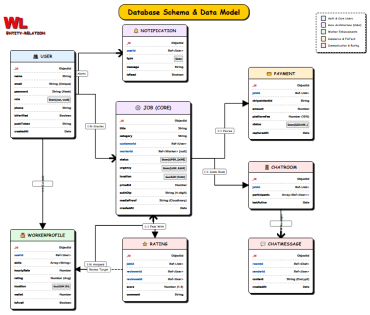

Fig. 3. Entity relationship design for transactional and communication modules.

C. Lifecycle and Trust Controls

Each job progresses through controlled states (e.g., `OPEN`, `ASSIGNED`, `IN_PROGRESS`, `COMPLETED`, `PAID`). OTP verification binds physical worker presence to digital state advancement. Completion proof via uploaded media supports dispute reduction and transparent auditability.

D. Matching and Dispatch Logic

Worker assignment combines skill compatibility, distance, and wait-time constraints. In cost-matrix terms, assignment can be solved through Hungarian optimization [1], [2]. The platform currently uses practical matching controllers aligned with this principle for efficient local dispatch.

IV. Methodology and Implementation

A. Frontend Layer

The frontend is implemented with React and Vite [11], with contextual state providers for authentication, notifications, and sockets. UI modules include job creation, dashboards, chat, map tracking, payment dialogs, and assistant widgets. Internationalization support enables English, Hindi, and Telugu interface flows using an i18n pipeline [10].

B. Backend Layer

The backend exposes domain-specific controllers (auth, job, lifecycle, matching, payment, chat, upload, notification, admin) and route groups. Middleware enforces authentication, error handling, and upload policies aligned with secure web engineering recommendations [12]. Socket events broadcast chat and status updates in near real time.

C. Payment and Event Handling

Payment authorization and completion are orchestrated using Stripe intents and webhook confirmation patterns [8], [9]. This prevents premature payment release and maintains consistency with job completion verification.

Geospatial filtering for worker discovery uses database-level location indexing and query support consistent with MongoDB geospatial capabilities [7].

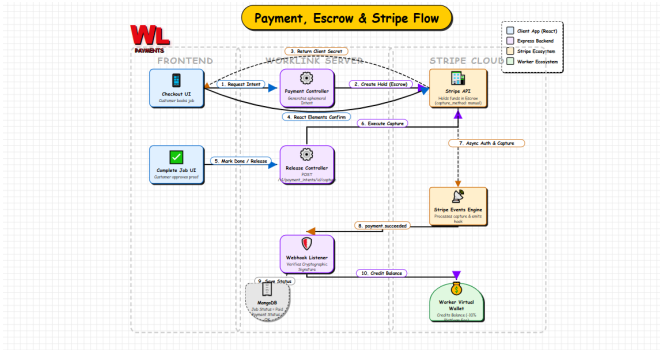

Fig. 4. Payment lifecycle with controlled release and status synchronization.

D. Notification and Communication Pipeline

The notification pipeline integrates persistent notifications and socket pushes for assignment updates, OTP events, travel updates, and completion milestones.

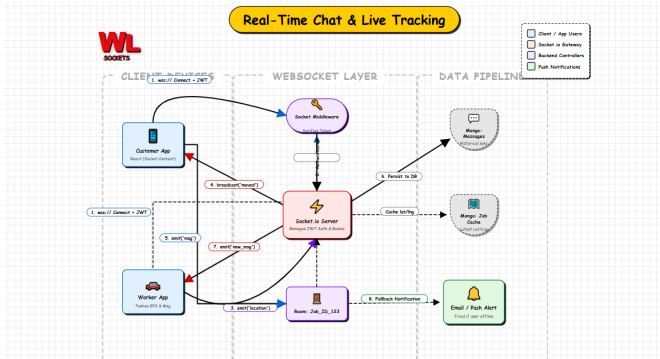

Fig. 5. Real-time notification flow across customer-worker interactions.

V. Results and Discussion

This project is validated through integrated engineering tests covering core transactional journeys rather than isolated algorithmic benchmarks. Validation scope includes registration and verification, job posting with media, worker assignment, OTP start, in-app communication, payment capture, completion proof, and final settlement.

A. Feature-Level Validation

Functional validation confirms end-to-end behavior for key modules listed in the implementation roadmap: job creation, OTP verification, chat, rating flow, email verification, payment integration, matching engine, and team formation support.

Real-time map tracking and advanced localization are included as iterative enhancement tracks.

B. UI Evaluation Snapshots (Recommended for Final Submission)

For a strong final-year research presentation, the most meaningful interface evidence should cover the complete service lifecycle. The recommended screenshot set is:

- **Job Creation Page:** shows structured requirement capture and media upload.
- **Worker Dashboard:** shows job visibility, assignment status, and action controls.
- **Chat/Assistant Page:** shows real-time communication and AI assistance.
- **Payment/OTP Verification Page:** shows trust checkpoint and transaction safety.

To insert these in Overleaf, upload your screenshots into `WorkLink/` and uncomment/adapt the below templates:

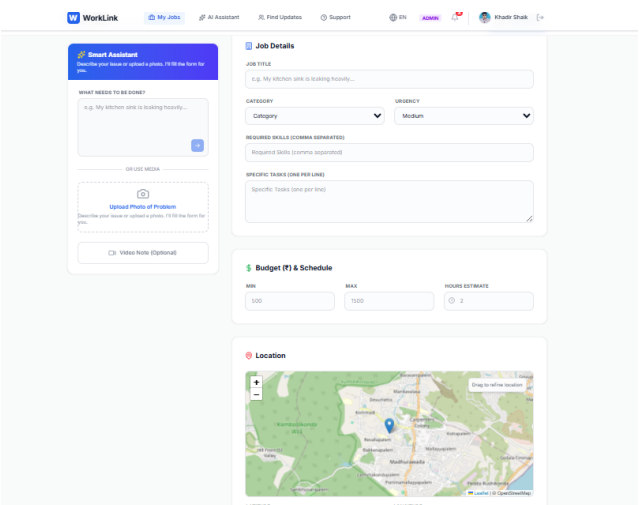

Fig. 6. Job creation interface with structured service inputs and media upload.

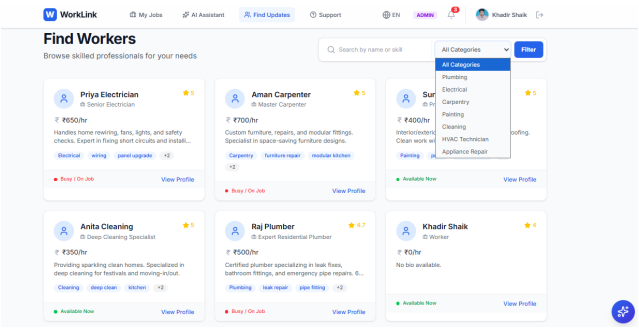

Fig. 7. Worker dashboard showing available jobs, assignment status, and action controls.

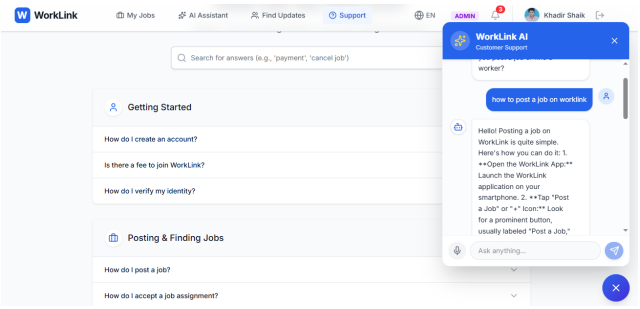

Fig. 8. Real-time chat interface with integrated AI assistant for guided communication and support.

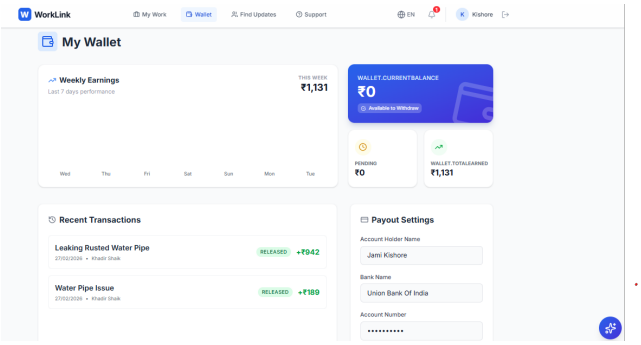

Fig. 9. OTP verification and payment workflow demonstrating trust checkpoint and transaction safety.

TABLE I
IMPLEMENTED MODULE COVERAGE

Module	Status	Validation Focus
Job Creation + Media	Completed	Upload, storage, retrieval
OTP Job Start	Completed	State transition integrity
AI Assistant	Completed	Contextual response behavior
Stripe Payments	Completed	Intent + webhook consistency
Matching Engine	Completed	Skill and availability alignment
Real-Time Chat	Completed	Message persistence + delivery
Ratings/Reviews	Completed	Post-completion feedback
Live Tracking	Completed/Iterative	Worker travel event sync
Multi-Language	Completed/Iterative	Locale switching coverage

C. Engineering Observations

The architecture provides clear operational advantages:

- **Transaction reliability:** Explicit lifecycle guards reduce ambiguous job states.
- **Low coordination friction:** Real-time chat and notifications reduce communication delays.

- **Trust reinforcement:** OTP validation and evidence-based completion improve confidence.
- **Worker enablement:** AI guidance and multilingual interface improve accessibility.

D. Limitations

Current limitations include dependency on stable connectivity for real-time updates, partial localization audit coverage across all pages, and limited large-scale production telemetry. These are engineering maturity constraints rather than architectural bottlenecks.

VI. Future Enhancements

Planned enhancements include:

- Full localization audit with domain-specific vernacular tuning.
- Offline-first worker mode with delayed synchronization for low-connectivity areas.
- Advanced map intelligence: geofencing confidence, ETA smoothing, and anomaly alerts.
- Explainable matching rationale to increase assignment transparency.
- Expanded admin analytics for dispute trend analysis and platform health metrics.
- Safety extensions including SOS escalation workflows and verified emergency channels.

VII. Conclusion

WorkLink demonstrates that a blue-collar service marketplace can be engineered as a reliable digital transaction system rather than a simple listing board. By combining lifecycle governance, secure payment orchestration, AI-assisted communication, and real-time collaboration, the platform addresses core trust and usability gaps in informal service ecosystems. The resulting system architecture is practical for final-year project deployment and extensible for production-grade evolution. This work provides a concrete, scalable foundation for inclusive and trustworthy service digitization.

Future work will focus on real-world validation with enterprise logs, adversarial robustness testing, SIEM integration, and explainable AI components using SHAP or LIME for analyst transparency.

References

[1] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[2] J. Munkres, "Algorithms for the assignment and transportation problems," *Journal of the Society for Industrial and Applied Mathematics*, vol. 5, no. 1, pp. 32–38, 1957.

[3] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[4] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference*, 1999.

[5] Google, "Gemini: A Family of Highly Capable Multimodal Models," arXiv:2312.11805, 2024.

[6] Socket.IO, "Socket.IO documentation," 2025. [Online]. Available: https://socket.io/docs/v4/

[7] MongoDB, "Geospatial queries and indexes," 2025. [Online]. Available: https://www.mongodb.com/docs/manual/geospatial-queries/

[8] Stripe, "Payment Intents API," 2025. [Online]. Available: https://docs.stripe.com/payments/payment-intents

[9] Stripe, "Using webhooks for asynchronous events," 2025. [Online]. Available: https://docs.stripe.com/webhooks

[10] i18next, "Internationalization framework for JavaScript," 2025. [Online]. Available: https://www.i18next.com/

[11] React Team, "React documentation," 2025. [Online]. Available: https://react.dev/

[12] OWASP Foundation, "OWASP Top 10: The ten most critical web application security risks," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/